Algoritmos e Lógica de Programação — Entrega 1

Aplicação das Estruturas Sequencial, de Decisão e de Repetição

Projeto:	Sistema Desktop para Gestão de Assinaturas de Jogos Educacionais — Messier Data & Creative
Disciplina:	Algoritmos e Lógica de Programação
Entrega:	Entrega 1 — Prazo: 06/04/2026
Curso:	Análise e Desenvolvimento de Sistemas — 1º Semestre 2026
Prof. Orientadora:	Professora Mary Luci
Integrantes:	João Pedro Teixeira dos Santos Raphael Carmo Mendes Daniel Bandini Grande Kauê Yudi Maeda Rodrigues Guilherme Koide Rangel

1. Introdução

Este documento apresenta a aplicação das estruturas fundamentais de algoritmos — sequencial, de decisão e de repetição — no contexto do Projeto Interdisciplinar Messier Data & Creative. O sistema é um aplicativo Desktop desenvolvido em C# para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo.

As estruturas apresentadas foram extraídas dos módulos centrais do sistema: o fluxo de login/validação de escola, o registro de acesso a um game e a exibição do catálogo disponível. Cada trecho de código é acompanhado da definição de entradas, saídas e observações reflexivas da equipe.

2. Estrutura Sequencial

A estrutura sequencial representa a execução de instruções em ordem linear, sem desvios. No projeto Messier, ela aparece em dois momentos principais: (a) na coleta e preparação dos dados de login da escola e (b) no registro de um acesso a game no banco de dados.

2.1 Fluxo Sequencial – Coleta de Dados de Login

Entradas	Saídas
Código da escola (string)	Objeto Escola carregado da base
Senha informada (string)	Boolean pacoteAtivo
IP de origem (string)	Boolean ipAutorizado

Trecho de código (estrutura sequencial):

```
// 1. Lê os campos informados pelo usuário
string codigoEscola = txtCodigoEscola.Text.Trim();
string senha = txtSenha.Text.Trim();
string ipOrigem = txtIpOrigem.Text.Trim();

// 2. Busca a escola no banco de dados
Escola escola = EscolaRepositorio.BuscarPorCodigo(codigoEscola);

// 3. Valida a senha via hash armazenado
bool senhaValida = SenhaHelper.Verificar(senha, escola.SenhaHash);

// 4. Verifica se o pacote da escola está ativo
bool pacoteAtivo = escola.Pacote != null && escola.Pacote.Ativo;

// 5. Verifica se o IP de origem consta na lista autorizada
bool ipAutorizado = escola.IpsAutorizados.Contains(ipOrigem);
```

Descrição: As cinco instruções acima executam em sequência estrita. Nenhuma depende do resultado de um desvio condicional — apenas leitura, consulta ao repositório e atribuição de variáveis booleanas que serão usadas nas próximas estruturas.

2.2 Fluxo Sequencial – Registro de Acesso a Game

Entradas	Saídas
Objeto Escola (com pacote e contador mensal)	Registro de Log inserido no banco
Objeto Game selecionado	Contador mensal incrementado
IP de origem	Mensagem de confirmação na tela
DateTime atual	

Trecho de código (estrutura sequencial):

```
// 1. Cria o objeto de log com os dados do acesso
LogAcesso log = new LogAcesso();
log.EscolaId = escola.Id;
log.GameId = gameSelecionado.Id;
log.IpOrigem = ipOrigem;
log.DataHora = DateTime.Now;
log.Resultado = "PERMITIDO";

// 2. Persiste o log na tabela LogAcesso
LogAcessoRepositorio.Inserir(log);

// 3. Incrementa o contador de acessos do mês para esta escola
escola.ConsumoMesAtual = escola.ConsumoMesAtual + 1;

// 4. Atualiza o consumo no banco de dados
EscolaRepositorio.AtualizarConsumo(escola.Id, escola.ConsumoMesAtual);

// 5. Exibe mensagem de sucesso ao usuário
lblStatus.Text = "Acesso registrado com sucesso!";
```

Descrição: Cada linha executa uma única tarefa específica em ordem definida. A criação do log precede a persistência; a persistência precede o incremento do contador; o incremento precede a atualização no banco. A inversão de qualquer passo causaria inconsistência nos dados.

3. Estrutura de Decisão

A estrutura de decisão redireciona o fluxo de execução com base em condições avaliadas em tempo de execução. No projeto Messier, o principal ponto de decisão é o processo de validação de login, que precisa verificar três condições independentes antes de conceder acesso à escola.

3.1 Validação de Login – Decisão em Cascata

Entradas	Saídas
senhaValida (bool)	Tela de catálogo (acesso liberado)
pacoteAtivo (bool)	ou Mensagem de erro específica (acesso negado)
ipAutorizado (bool)	

Trecho de código (estrutura de decisão):

```
// Decisão 1: verifica se a senha está correta
if (senhaValida == false)
{
lblMensagem.Text = "Credenciais inválidas. Verifique o código e a senha.";
return;
}

// Decisão 2: verifica se a escola possui pacote ativo
if (pacoteAtivo == false)
{
lblMensagem.Text = "Escola sem pacote ativo. Entre em contato com a Messier.";
return;
}

// Decisão 3: verifica se o IP de origem está na lista autorizada
if (ipAutorizado == false)
{
LogAcessoRepositorio.RegistrarBloqueio(escola.Id, ipOrigem, DateTime.Now);
lblMensagem.Text = "Acesso bloqueado: IP de origem não autorizado.";
return;
}

// Se todas as condições foram satisfeitas, abre o catálogo
AbrirTelaCatalogo(escola);
```

Descrição: Cada bloco *if* avalia uma condição distinta e retorna imediatamente ao encontrar a primeira falha, emitindo uma mensagem específica. Isso evita verificações desnecessárias e fornece ao usuário um diagnóstico preciso do motivo do bloqueio. Somente quando as três condições são verdadeiras o fluxo prossegue para a tela de catálogo.

3.2 Decisão no Registro de Acesso – Limite Mensal

Entradas	Saídas
escola.ConsumoMesAtual (int)	Acesso permitido + log PERMITIDO
escola.Pacote.LimiteAcessosMes (int)	ou Acesso bloqueado + log BLOQUEADO + mensagem

Trecho de código (decisão simples com else):

```
// Verifica se o limite mensal de acessos foi atingido
if (escola.ConsumoMesAtual >= escola.Pacote.LimiteAcessosMes)
{
// Registra a tentativa como BLOQUEADA no log
LogAcesso logBloqueado = new LogAcesso();
logBloqueado.EscolaId = escola.Id;
logBloqueado.GameId = gameSelecionado.Id;
logBloqueado.IpOrigem = ipOrigem;
logBloqueado.DataHora = DateTime.Now;
logBloqueado.Resultado = "BLOQUEADO";
LogAcessoRepositorio.Inserir(logBloqueado);
lblStatus.Text = "Limite mensal de acessos atingido. "
+ "Contate a Messier para ampliar seu pacote.";
}
else
{
// Acesso dentro do limite: registra e incrementa o contador
LogAcesso logPermitido = new LogAcesso();
logPermitido.EscolaId = escola.Id;
logPermitido.GameId = gameSelecionado.Id;
logPermitido.IpOrigem = ipOrigem;
logPermitido.DataHora = DateTime.Now;
logPermitido.Resultado = "PERMITIDO";
LogAcessoRepositorio.Inserir(logPermitido);
escola.ConsumoMesAtual = escola.ConsumoMesAtual + 1;
EscolaRepositorio.AtualizarConsumo(escola.Id, escola.ConsumoMesAtual);
lblStatus.Text = "Game acessado com sucesso!";
}
```

Descrição: O bloco if/else avalia se o número atual de acessos já alcançou o limite contratado. Em ambos os caminhos, um registro de log é gerado — garantindo rastreabilidade completa —, mas apenas no caminho else o contador é incrementado e o acesso é efetivamente liberado.

4. Estrutura de Repetição

A estrutura de repetição executa um bloco de código múltiplas vezes enquanto uma condição for satisfeita ou para cada elemento de uma coleção. No projeto Messier, ela aparece na exibição do catálogo de games e na geração de relatórios.

4.1 Exibição do Catálogo – Iteração sobre a Lista de Games

Entradas	Saídas
Lista de objetos Game (List<Game>) associados ao pacote da escola	Linhas exibidas no componente DataGridView (um item por game)
	Mensagem se catálogo vazio

Trecho de código (estrutura de repetição – foreach):

```
// Busca os games disponíveis no pacote ativo da escola
List<Game> gamesDisponiveis = GameRepositorio.BuscarPorPacote(escola.Pacote.Id);

// Limpa a lista antes de popular
dgvCatalogo.Rows.Clear();

// Percorre cada game da lista e adiciona uma linha na grade
foreach (Game game in gamesDisponiveis)
{
string nomeGame = game.Nome;
string disciplina = game.Disciplina;
string faixaEtaria = game.FaixaEtaria;
string statusGame = game.Ativo ? "Disponível" : "Indisponível";

dgvCatalogo.Rows.Add(nomeGame, disciplina, faixaEtaria, statusGame);
}

// Decisão após a repetição: informa se o catálogo está vazio
if (dgvCatalogo.Rows.Count == 0)
{
lblCatalogo.Text = "Nenhum game disponível no pacote atual.";
}
```

Descrição: O *foreach* itera sobre cada objeto *Game* retornado pelo repositório. A cada iteração, os atributos do game são extraídos e uma nova linha é adicionada ao componente visual. O número de iterações é igual ao número de games no pacote — podendo ser zero, caso em que a decisão pós-repetição exibe a

mensagem de catálogo vazio.

4.2 Geração de Relatório – Iteração sobre Registros de Log

Entradas	Saídas
Lista de LogAcesso filtrada por escola e mês/ano (List<LogAcesso>)	Contador de acessos PERMITIDOS
	Contador de acessos BLOQUEADOS
	Total geral
	Tabela de relatório preenchida

Trecho de código (estrutura de repetição – while):

```
// Busca os logs da escola no mês/ano selecionado
List<LogAcesso> logs = LogAcessoRepositorio.BuscarPorEscolaMes(
escola.Id, mesSelecionado, anoSelecionado);

// Inicializa contadores antes do laço
int totalPermitidos = 0;
int totalBloqueados = 0;
int indice = 0;

// Percorre os registros enquanto houver itens na lista
while (indice < logs.Count)
{
LogAcesso registro = logs[indice];

if (registro.Resultado == "PERMITIDO")
{
totalPermitidos = totalPermitidos + 1;
}
else
{
totalBloqueados = totalBloqueados + 1;
}

dgvRelatorio.Rows.Add(
registro.DataHora.ToString("dd/MM/yyyy HH:mm"),
registro.NomeGame,
registro.IpOrigem,
registro.Resultado
);

indice = indice + 1;
}

// Exibe os totais calculados
lblPermitidos.Text = "Acessos permitidos: " + totalPermitidos.ToString();
lblBloqueados.Text = "Acessos bloqueados: " + totalBloqueados.ToString();
lblTotal.Text = "Total no período: " + (totalPermitidos + totalBloqueados).ToString();
```

Descrição: O laço *while* usa um índice inteiro para percorrer a lista de logs. A cada passo, a condição é avaliada, o registro corrente é processado, os contadores são acumulados e o índice é incrementado manualmente. A estrutura é encerrada quando o índice atinge o tamanho total da lista, garantindo que todos os registros sejam processados exatamente uma vez.

5. Observações Reflexivas do Grupo

• Integração natural entre as estruturas

Ao implementar o módulo de login, o grupo percebeu que as três estruturas se complementam de forma indissociável: a coleta dos dados segue o fluxo sequencial, a validação exige decisões encadeadas e o catálogo de games só pode ser exibido após iterar sobre a lista retornada pelo banco. Nenhuma das estruturas, isolada, seria suficiente para completar um único caso de uso do sistema.

• Importância da ordem sequencial

Um erro inicial do grupo foi tentar incrementar o contador de acessos antes de inserir o log. Isso gerava inconsistências: se a inserção do log falhasse, o contador já estaria atualizado. A revisão do fluxo sequencial — log primeiro, contador depois — corrigiu o problema e reforçou que a ordem das instruções sequenciais tem impacto direto na integridade dos dados.

• Mensagens de erro específicas nas decisões

Na primeira versão do login, havia apenas uma mensagem genérica ('Acesso negado'). A revisão das estruturas de decisão levou o grupo a separar os blocos por condição (senha inválida, pacote inativo, IP bloqueado), produzindo mensagens específicas. Isso melhorou a usabilidade e facilita a auditoria de tentativas de acesso.

• Rastreabilidade garantida pela repetição

O laço de geração de relatório evidenciou a necessidade de registrar tanto acessos permitidos quanto bloqueados. Sem iterar sobre todos os logs, a equipe teria subestimado o consumo real e não detectaria padrões de tentativas de acesso por IPs não autorizados. A estrutura de repetição tornou visível um requisito de rastreabilidade que antes era implícito.

• Possíveis melhorias identificadas

O grupo identificou que, em cenários com grande volume de logs, o laço while carregando todos os registros em memória pode ser lento. Uma melhoria futura seria paginar a consulta ao banco, aplicando filtros diretamente no SQL e reduzindo o número de iterações. Essa observação nasceu justamente da análise detalhada da estrutura de repetição durante a elaboração deste documento.

6. Conclusão

A análise das estruturas algorítmicas aplicadas ao projeto Messier demonstrou que algoritmo e lógica de programação não são disciplinas abstratas: cada decisão de implementação — a ordem de uma instrução, o critério de uma condição ou a forma de um laço — tem reflexo direto na corretude, na usabilidade e na rastreabilidade do sistema.

O grupo concluiu que a documentação explícita dessas estruturas, separando entradas, saídas e observações, é uma prática de engenharia valiosa: ela expõe falhas de lógica antes que se tornem bugs em produção e serve de base para revisões e melhorias incrementais ao longo das próximas entregas do PI.

@fecap
(11) 3272-2222 / www.fecap.br
Av. Liberdade, 532 / 01502-001 / São Paulo - SP
Fundação Escola de Comércio Álvares Penteado - FECAP